SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

Parte 1 de 4 — Aceleradores de Hardware y Arquitectura de GPUs

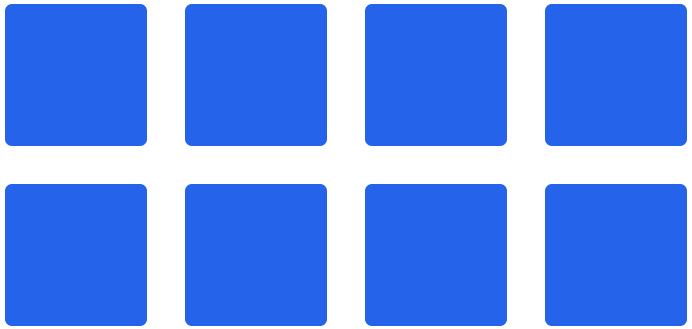
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

Parte 1 de 4 — Aceleradores de Hardware y Arquitectura de GPUs

CPU frente a GPU en Cómputo de IA

De núcleos complejos optimizados para latencia secuencial a miles de núcleos paralelos SIMT

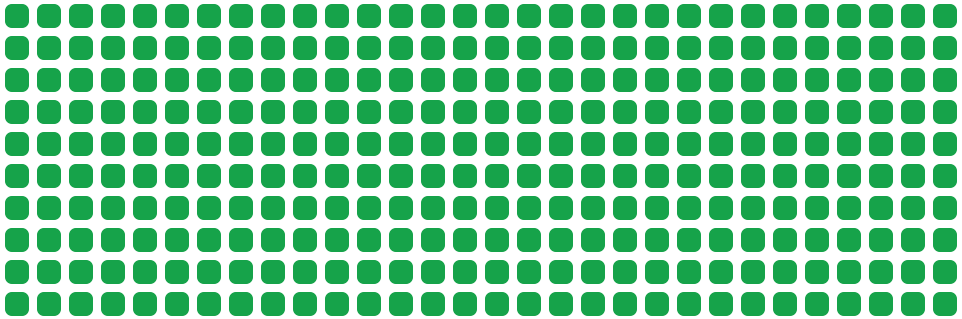
CPU: pocos núcleos complejos



16 a 64 núcleos, cachés grandes (L2/L3)

Rápida en una secuencia: control de flujo y SO

GPU: miles de núcleos simples



Miles de núcleos SIMT, cachés pequeñas

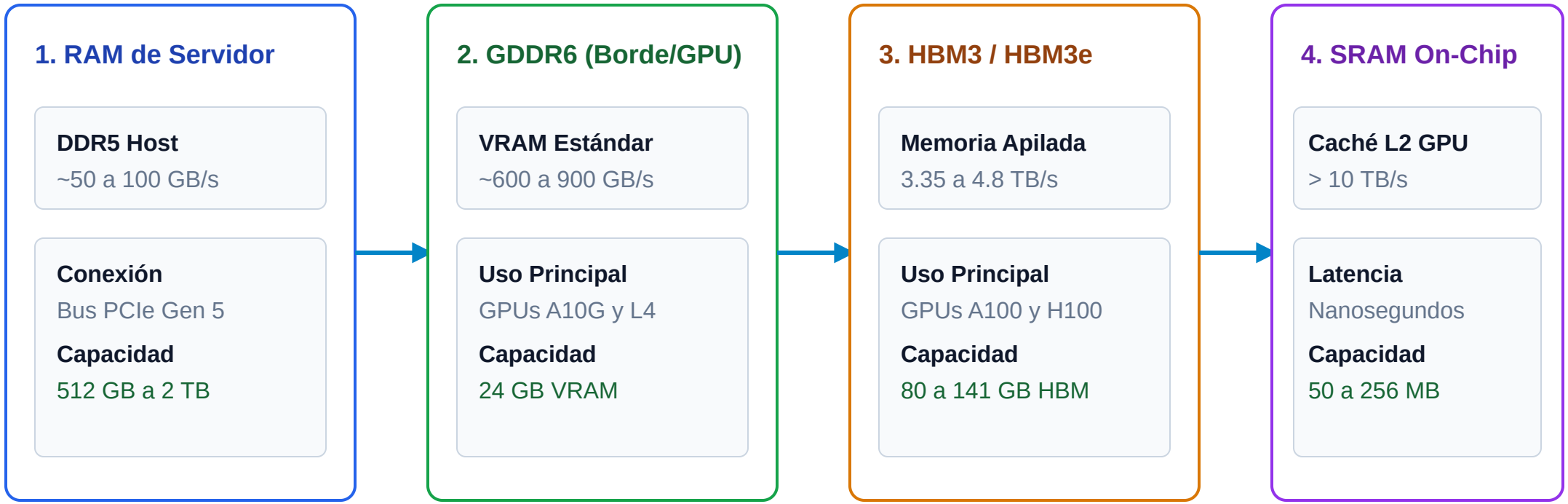
Muchas multiplicaciones de matrices a la vez

Inferir un LLM es multiplicar matrices: gana quien hace más operaciones en paralelo

Fuente: NVIDIA Developer — GPU Architecture Overview (2025)

Jerarquía de Memoria en Aceleradores de IA

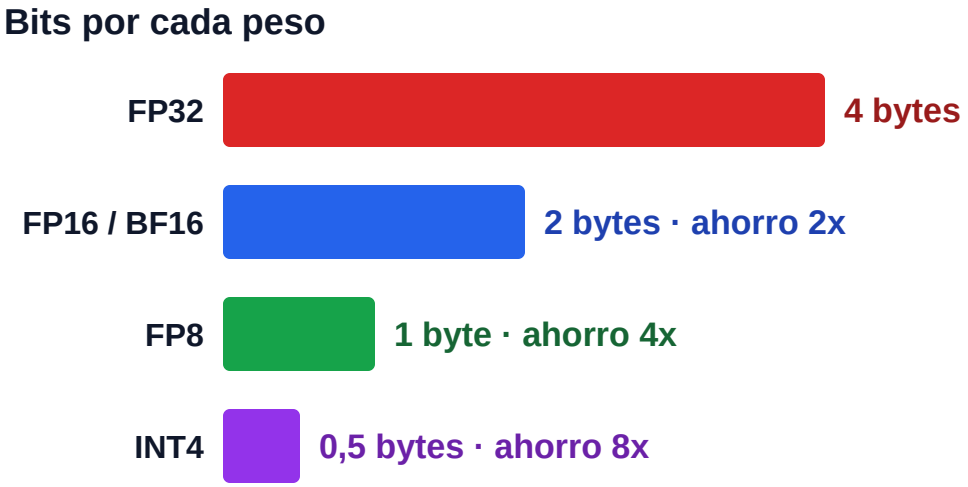
El verdadero cuello de botella en inferencia es transferir datos de memoria al silicio

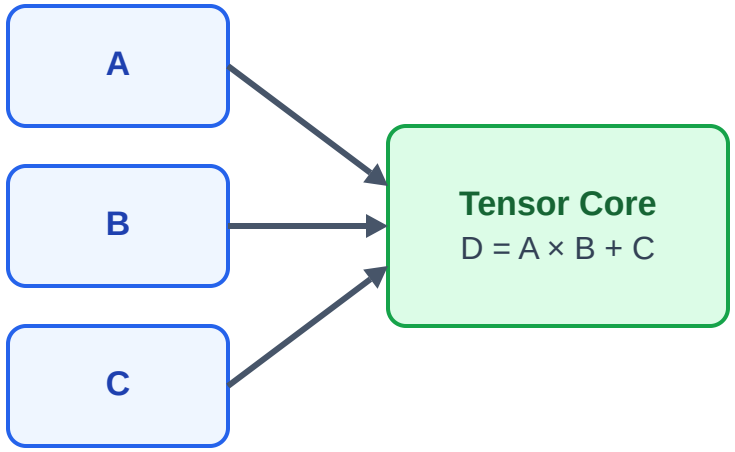


Fuente: JEDEC — HBM3 Standard JESD238A & Micron Technology (2025)

Tensor Cores y Formatos Numéricos

De precisión simple FP32 a formatos especializados FP16, BF16 y FP8 para maximizar rendimiento





Una operación matricial en un ciclo de reloj

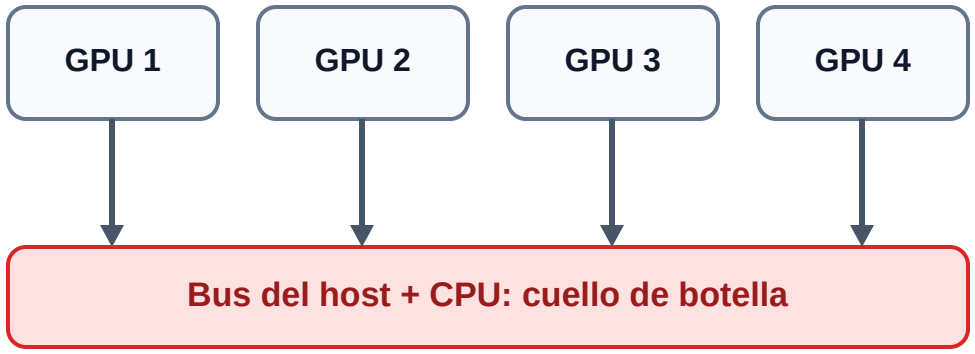
Menos bits por peso y hardware matricial: más cálculo con la misma memoria

Fuente: IEEE Micro — NVIDIA Hopper Tensor Core Architecture (2024)

Interconexiones Multi-GPU: NVLink frente a PCIe

El paralelismo de tensores requiere comunicación inter-GPU sin pasar por el bus del procesador

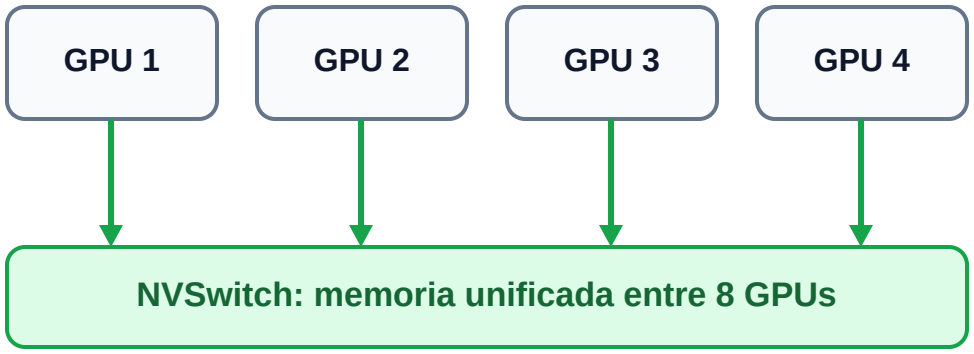
PCIe Gen 5: bus compartido



64 GB/s por dirección

Sirve para modelos de un solo nodo GPU

NVLink + NVSwitch: punto a punto



900 GB/s a 1,8 TB/s por GPU

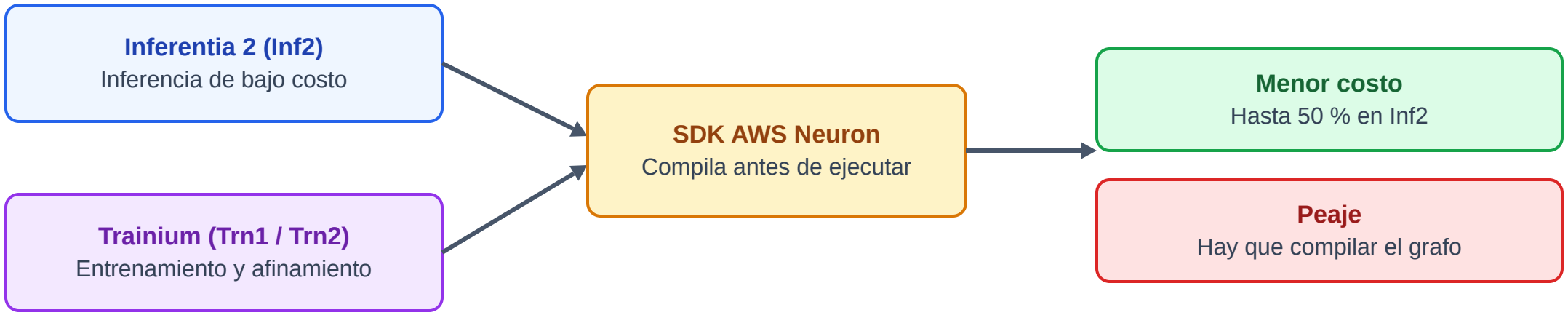
Obligatorio con paralelismo tensorial (70B a 405B)

Si dos GPU deben conversar en cada token, el enlace define la latencia

Fuente: NVIDIA — NVLink and NVSwitch Architecture Specification (2024)

Silicio Propietario de Nube: AWS Inferentia y Trainium

Alternativas a GPUs comerciales optimizadas para costo y rendimiento en cargas de trabajo de IA



PyTorch / Hugging Face entran al compilador

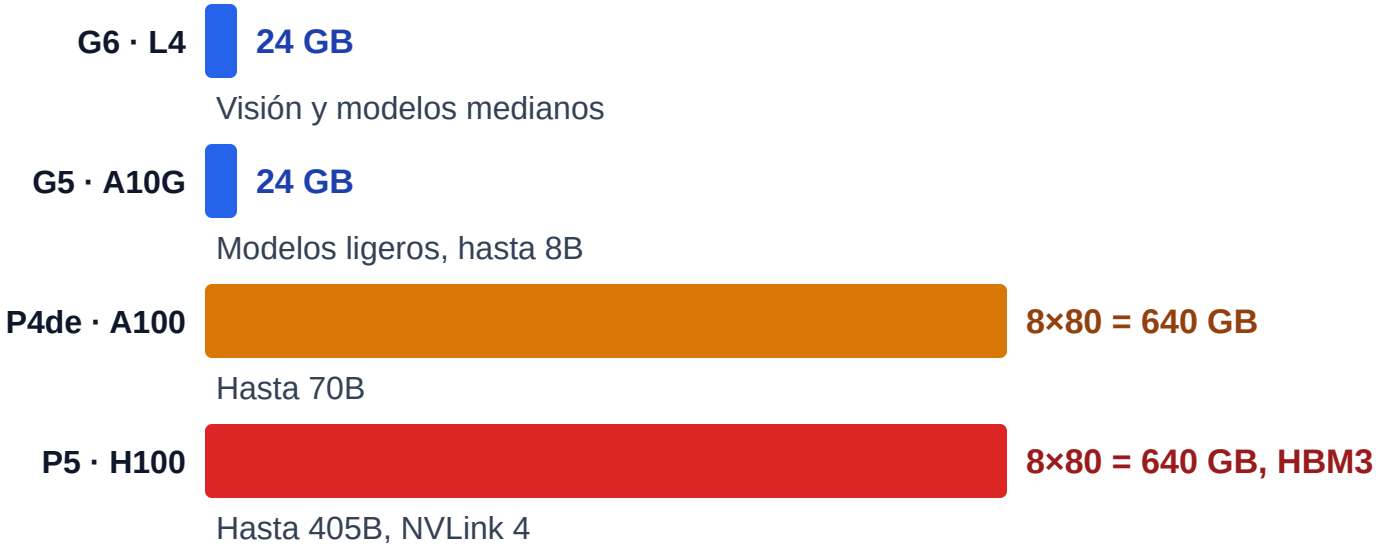
Silicio propio: se paga menos a cambio de atarse al compilador

Fuente: AWS Documentation — Annapurna Labs NeuronCore Overview (2025)

Familias de Instancias GPU en AWS EC2

Selección del tamaño de máquina adecuado según el tamaño del modelo y el volumen de tráfico

Memoria GPU por instancia: la familia se elige por el tamaño del modelo



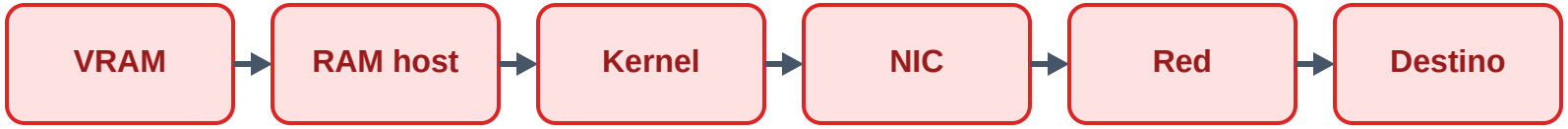
Precio on-demand us-east-1 (nov 2026): g5.xlarge $1,006/h · p5.48xlarge $55,04/h

Fuente: AWS Documentation — Amazon EC2 Accelerated Computing Instances (2025)

Redes de Ultra-Baja Latencia: EFA y RoCEv2

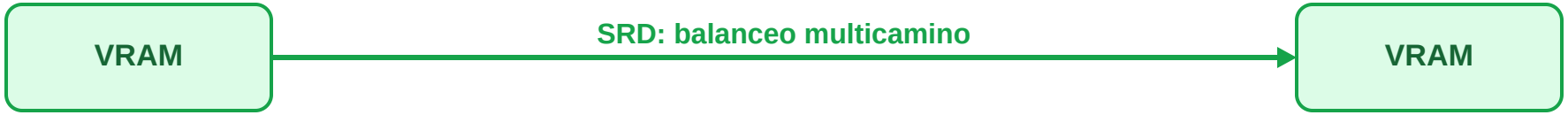
Transferencia directa de memoria entre servidores eliminando la sobrecarga del kernel del SO

TCP/IP clásico: copias e interrupciones en cada salto



Resultado: latencia impredecible, se rompen los SLA interactivos

EFA + RDMA: de VRAM a VRAM sin pasar por la CPU



Resultado: latencia sub-microsegundo y throughput estable

Saltarse el sistema operativo es lo que hace viable la inferencia distribuida

Fuente: AWS Whitepaper — High Performance Computing on AWS & EFA Architecture (2024)

Particionamiento de GPU: NVIDIA Multi-Instance GPU (MIG)

División por hardware de una GPU de 80GB en hasta 7 instancias independientes con SLAs garantizados

NVIDIA A100 / H100 (80 GB VRAM Física) Dividida con MIG

MIG 1g.10gb

1 Slice Cómputo (14 SM)

10 GB VRAM Aislada

Uso: Modelo Embeddings

MIG 1g.10gb

1 Slice Cómputo (14 SM)

10 GB VRAM Aislada

Uso: Reranking Cross-Enc

MIG 2g.20gb

2 Slices Cómputo (28 SM)

20 GB VRAM Aislada

Uso: LLM Llama-3-8B

MIG 3g.40gb

3 Slices Cómputo (42 SM)

40 GB VRAM Aislada

Uso: Inferencia Batch 14B

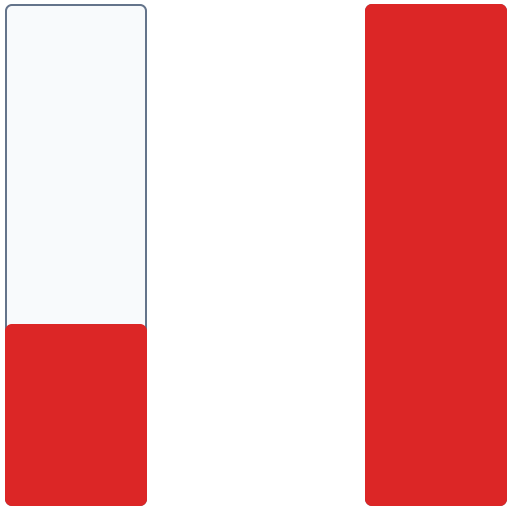
Aislamiento Físico: Canales de memoria, decodificadores y cachés L2 completamente dedicados

Fuente: NVIDIA — Multi-Instance GPU User Guide (2025)

Eficiencia Energética y Refrigeración en Centros de Datos

Densidades de más de 40 kW por rack exigen transición de aire forzado a refrigeración líquida directa

Aire: 10 a 15 kW por rack



Límite

Un servidor 8×H100
> 10,2 kW

→ **throttling**

Líquido directo: 40 a más de 100 kW



Capacidad

PUE ≈ 1,15
Líquido

PUE ≈ 1,5
Aire

La densidad de GPU obliga a rediseñar el datacenter, no solo el servidor

Fuente: Uptime Institute — Data Center Cooling & AI Power Density Trends (2025)

GPU Passthrough en Entornos OpenStack

Asignación directa de dispositivos PCI desde el hipervisor KVM a máquinas virtuales de cómputo

1. Host Físico (KVM / IOMMU)

Módulo VFIO-PCI
Desvincula la GPU del driver del host

Grupos IOMMU
Aislamiento seguro de memoria DMA

Configuración Nova
pci.passthrough_whitelist

2. Planificador de Nova

Filtro PCI en Scheduler
Ubica nodos con tarjetas libres

Flavor Especializado
extra_specs pci_passthrough:alias

Asignación Exclusiva
No admite sobreventa de hardware

3. Máquina Virtual Invitada

Visibilidad Nativa PCI
La VM ve el hardware de la tarjeta

Drivers NVIDIA Propios
Instala CUDA y vLLM sin parches

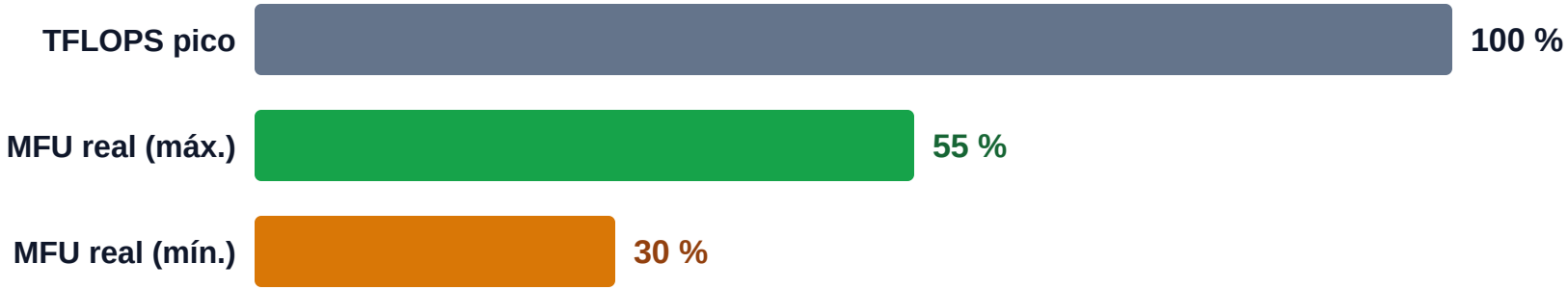
Rendimiento Bare-Metal
Pérdida menor al 1 % frente a físico

Fuente: OpenStack Docs — PCI Passthrough and SR-IOV Configuration (2025)

Rendimiento Real frente a Teórico: MFU y Throughput

Diferencia entre FLOPS pico en hoja técnica y la utilización efectiva real del silicio en inferencia

De la hoja técnica a la realidad



¿Dónde se pierde el resto?

Decodificación
Leer pesos para cada token: limitada por memoria

Burbujas en el lote
Peticiones de longitud dispar

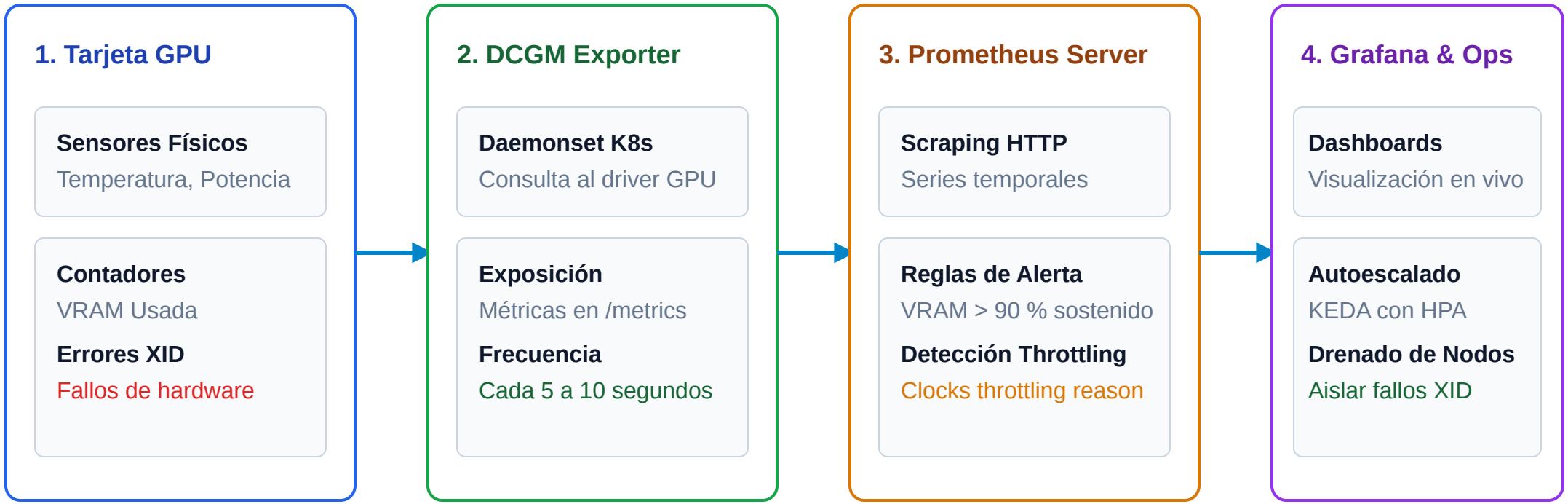
Copias host a GPU
Prompts por el bus PCIe

Dimensionar con el pico teórico deja el clúster corto

Fuente: Chowdhery et al. — PaLM: Scaling Language Modeling with Pathways (MFU Definition) (2024)

Monitorización de GPUs con NVIDIA DCGM

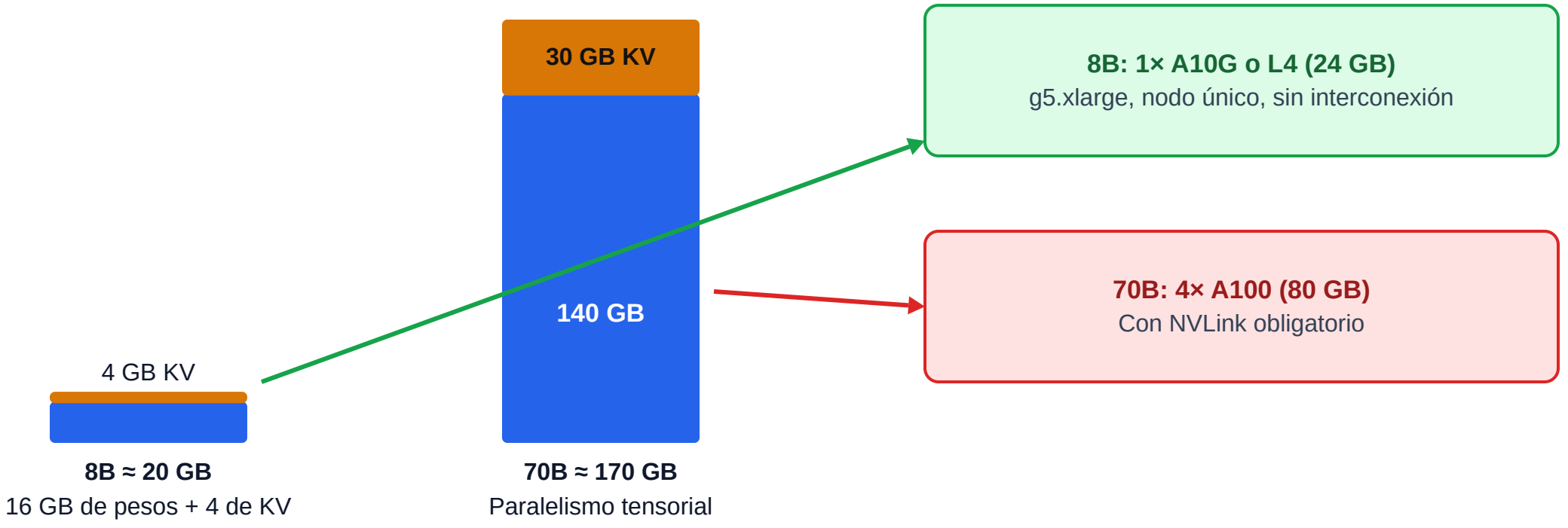
Métricas esenciales para detectar saturación de memoria, sobrecalentamiento y estrangulamiento térmico



Fuente: NVIDIA — Data Center GPU Manager (DCGM) User Guide (2025)

Caso Ilustrativo: Selección de Hardware para Inferencia

Caso ilustrativo de arquitectura, no es una empresa real — Matriz de decisión por tamaño de modelo



La memoria requerida decide el hardware

Fuente: AWS Architecture Center — Generative AI Sizing Reference Pattern (2025)

SÍNTESIS CONCEPTUAL

Principios Clave de Cómputo Acelerado

Cuatro reglas fundamentales para el diseño de infraestructura física y en la nube para IA

1. Ancho de Banda

Memoria HBM3

La tasa de transferencia es el verdadero cuello de botella durante la decodificación

> 3 TB/s en A100/H100

2. Precisión Numérica

Formatos FP16 / FP8

Reducir los bits por peso duplica o multiplica la capacidad efectiva de la memoria VRAM

2x a 4x Throughput

3. Interconexión

NVLink y EFA

Bypass del bus PCIe y del kernel del SO para sincronización de tensores ultra-baja

Latencia sub-microseg

4. Particionamiento

NVIDIA MIG

División por hardware para evitar vecinos ruidosos y optimizar inversión en servidores

Hasta 7 VMs por GPU

SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

Parte 2 de 4 — Cuantización y Optimización de Modelos

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

Parte 2 de 4 — Cuantización y Optimización de Modelos

Presupuesto de Memoria VRAM en Inferencia

Desglose estricto de componentes para evitar errores de falta de memoria (Out-Of-Memory)

Una GPU de 24 GB: qué ocupa la memoria



Fórmula: bytes por parámetro × parámetros

Ejemplo FP16: 8B × 2 bytes ≈ 16 GB

La KV-Cache depende de contexto y concurrencia

Activaciones: picos durante el prefill

Driver CUDA: 1 a 2 GB reservados

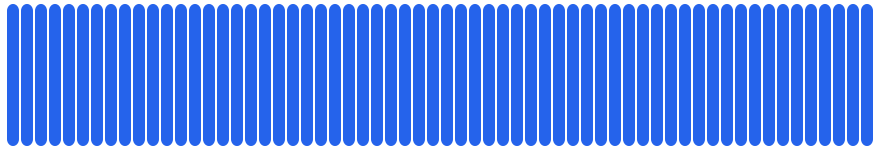
Regla FinOps: no superar el 90 % de la VRAM

Fuente: HuggingFace Documentation — Model Memory Estimation Guide (2025)

Fundamentos de Cuantización: De FP16 a Enteros

Mapeo de valores continuos de coma flotante a una cuadrícula discreta de menor resolución

FP16: 65.536 valores por peso



→

INT4: 16 niveles



Escala y punto cero: cada peso se redondea al nivel más cercano





Menos niveles, menos memoria: un modelo de 70B baja de 140 a 40 GB

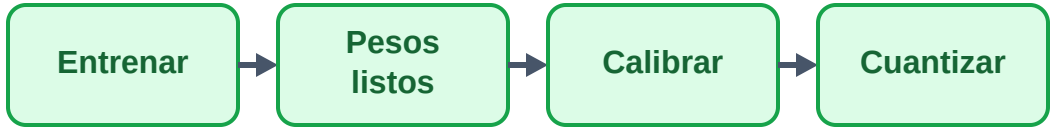
Fuente: Gholami et al. — A Survey of Quantization Methods for Efficient Neural Network Inference (2024)

Post-Training Quantization (PTQ) vs QAT

Compresión directa de modelos preentrenados frente a entrenamiento consciente de la cuantización

¿Cuándo se cuantiza?

PTQ



Minutos a pocas horas
en una sola GPU

QAT

Entrenar simulando el redondeo en cada paso

Días de cómputo
y datos masivos

En producción domina PTQ (AWQ, GPTQ); QAT queda para menos de 3 bits

Fuente: Jacob et al. — Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference (2024)

Algoritmo GPTQ: Compensación de Error por Capas

Ajuste dinámico de los parámetros no cuantizados para neutralizar la pérdida de precisión

1. Calibración Inicial

Muestra de Calibración
128 a 512 secuencias de texto

Activaciones de Capa
Calcula la curvatura de error

Sin Reentrenamiento
Proceso puramente analítico

2. Cuantización por Bloques

Proyección Columna a Columna
Cuantiza columnas a 4 bits

Cálculo del Residuo
Mide la diferencia con FP16

Compensación Inmediata
Ajusta los pesos restantes

3. Resultado Final

Modelo Comprimido a INT4
Tamaño reducido 4x

Preservación de Capacidad
> 99 % de retención de precisión

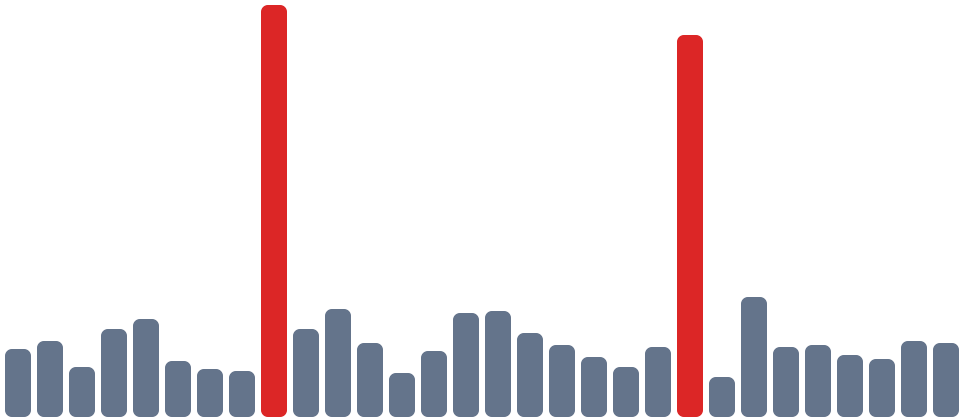
Ejecución en GPU
Desempaque rápido en Tensor Cores

Fuente: Frantar et al. — GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers (2023)

Algoritmo AWQ: Protección de Canales Salientes

El 1 % de los parámetros concentra la inteligencia del modelo y debe protegerse de la compresión

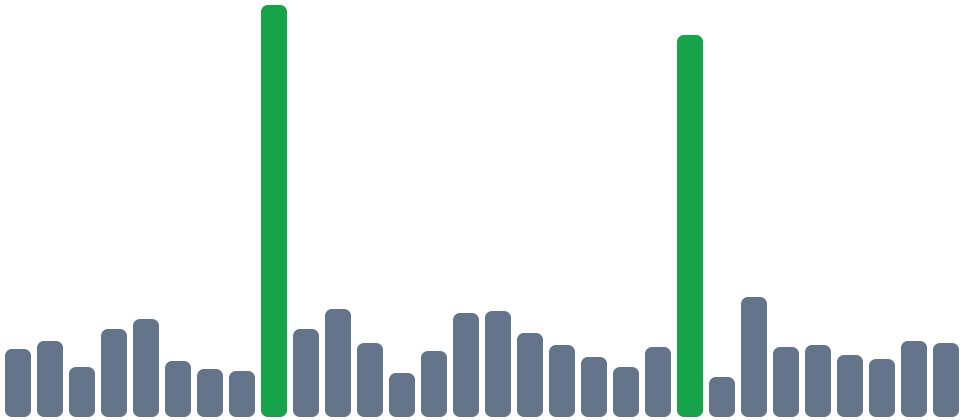
Uniforme: todos con la misma escala



Activaciones por canal; el 1 % domina

Redondearlo igual degrada razonamiento y lógica

AWQ: protege los canales salientes



Mide activaciones reales y escala los críticos

Más velocidad que GPTQ en vLLM

No todos los pesos importan igual: la precisión se gasta donde cuenta

Fuente: Lin et al. — AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration (2024)

Formato GGUF y Ecosistema llama.cpp

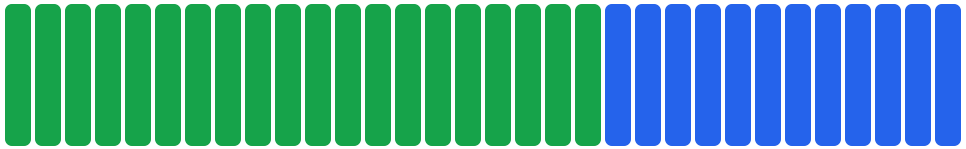
Inferencia híbrida dividiendo capas del modelo entre CPU de servidor y memoria de tarjeta gráfica

Archivo .gguf único
Metadatos + tokenizador + pesos

mmap: carga sin deserializar

Cuantización mixta: Q4_K_M, Q5_K_M

División flexible de capas



20 capas en GPU **12 en CPU**

C/C++ nativo, sin Python

Destino: borde (edge), PC de desarrollo y Ollama

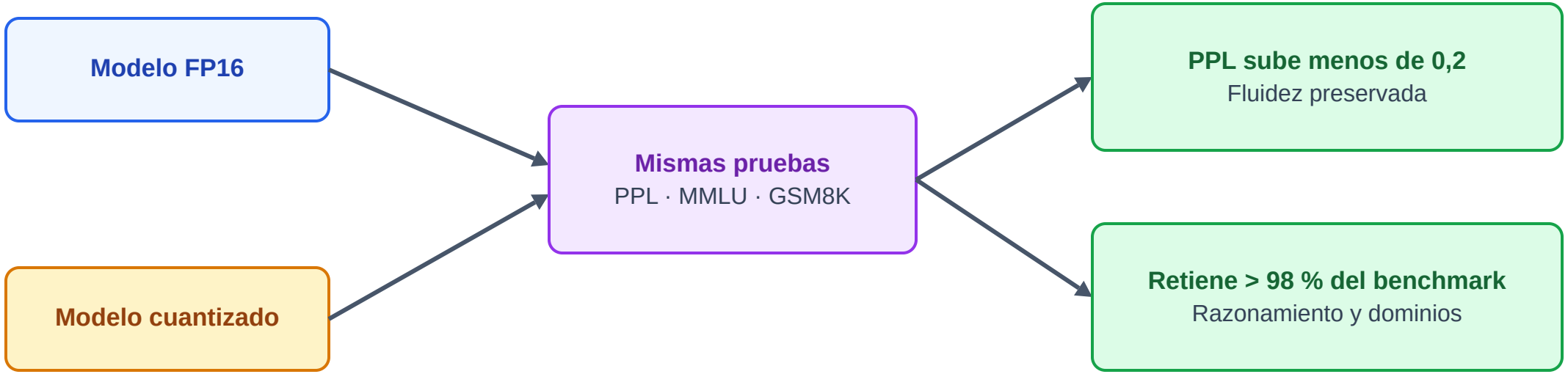
GGUF cambia rendimiento máximo por portabilidad

Fuente: Gerganov et al. — llama.cpp & GGUF Binary Specification (2025)

Métricas de Degradación: Perplejidad y Benchmarks

Evaluación rigurosa de la pérdida de calidad sintáctica y de razonamiento tras la compresión

Prueba de aceptación antes de desplegar un modelo cuantizado



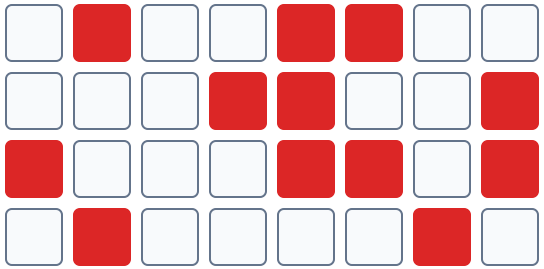
GSM8K es sensible a errores numéricos: mirar solo PPL no basta

Fuente: Hendrycks et al. — Measuring Massive Multitask Language Understanding (MMLU) (2024)

Poda (Pruning) y Dispersión Estructurada 2:4

Aceleración por hardware eliminando pesos redundantes en bloques regulares de matrices

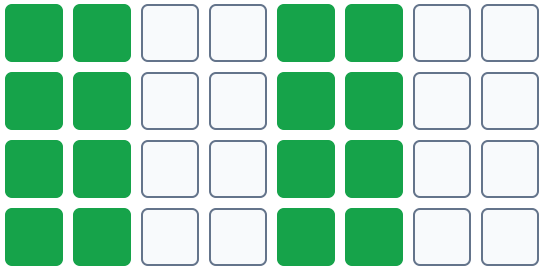
No estructurada: ceros al azar



Patrón irregular: rompe la ejecución SIMT

Ahorra disco, no acelera

2:4: dos de cada cuatro en cero



Patrón regular que el hardware reconoce

Hasta 2x de throughput

Azul claro = peso en cero; color = peso conservado

Podar solo ayuda si el silicio sabe saltarse los ceros

Fuente: NVIDIA Developer — Structured Sparsity in Ampere and Hopper Architecture (2024)

Destilación de Conocimiento: Profesor a Estudiante

Transferencia de la distribución probabilística de un modelo masivo a un modelo compacto

1. Modelo Profesor (Teacher)

Gran Escala (70B a 405B)
Razonamiento y alta capacidad

Genera 'Soft Labels'
Probabilidades continuas

Temperatura Elevada
Revela relaciones semánticas sutiles

2. Proceso de Transferencia

Alineación de Probabilidades
Minimiza divergencia KL de salida

Entrenamiento Eficiente
Aprende qué es probable y qué no

Datos Sintéticos
Generados por el propio profesor

3. Modelo Estudiante (Student)

Escala Compacta (1B a 8B)
Opera en una sola GPU económica

Velocidad de Inferencia
5x a 10x mayor generación de tokens

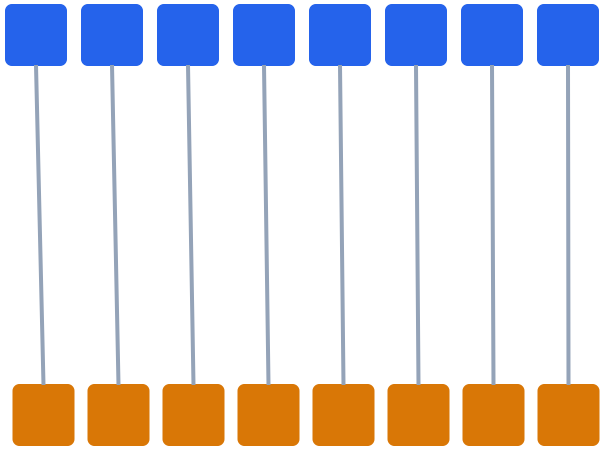
Eficiencia en la Nube
Costo operativo mucho menor

Fuente: Hinton et al. — Distilling the Knowledge in a Neural Network & TinyLlama Project (2024)

Evolución de la Atención: MHA, MQA y GQA

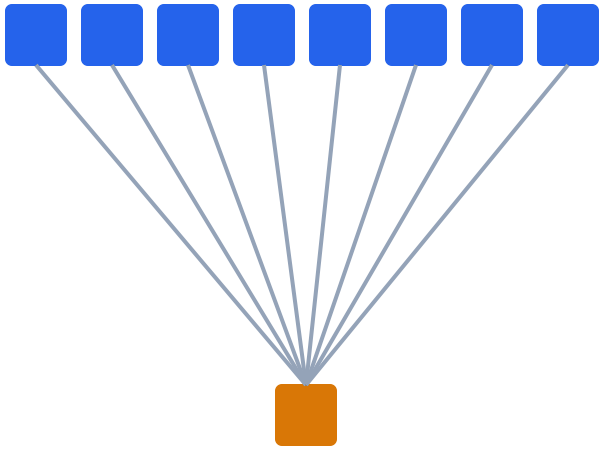
Reducción drástica del tamaño de la memoria KV-Cache compartiendo cabezas de claves y valores

MHA 1:1:1



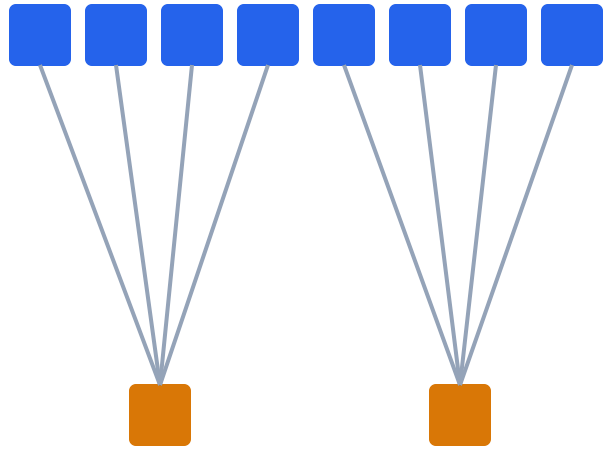
KV-Cache 8x a 32x mayor

MQA N:1:1



Máxima compresión, baja calidad

GQA agrupada



8x de ahorro sin perder precisión

Azul: consultas (Q) · Amarillo: pares Key/Value en caché

Compartir claves reduce la caché: GQA es el equilibrio habitual

Fuente: Ainslie et al. — GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints (2023)

Cuantización de la KV-Cache: FP8 e INT8

Compresión en tiempo real de los estados pasados de la conversación en contextos de más de 32k tokens

KV-Cache FP16 · GPU de 24 GB

2 GB por usuario con 32k tokens: se llena



Pocas consultas y luego OOM

KV-Cache FP8

1 byte por valor | Libre



El doble de usuarios, mismo hardware

Activación en el motor
--kv-cache-dtype fp8

Duplicar concurrencia sin comprar GPU: de 2 bytes a 1 byte por valor

Fuente: vLLM Documentation — KV-Cache Quantization (FP8 and INT8) Benchmarks (2025)

Compiladores de Inferencia: NVIDIA TensorRT-LLM

Fusión de operaciones matemáticas y optimización de kernels a nivel de ensamblador de GPU

1. Grafo PyTorch Estándar

Operaciones Aisladas
Muchos lanzamientos de kernels

Sobrecarga de Memoria
Guarda tensores en VRAM lenta

Latencia Adicional
GPU en espera de memoria VRAM

2. Fusión de Kernels

Fusión de Operaciones
Norm + MatMul en un solo kernel

Cómputo en Registros
Mantiene los datos en SRAM rápida

FlashAttention Integrado
Atención sin materializar matrices

3. Motor Optimizado

Binario Nativo GPU
Genera plan de ejecución compilado

Rendimiento Extremo
Hasta 2.5x menor latencia por token

Consideración
Binario atado a la GPU destino

Fuente: NVIDIA Developer — TensorRT-LLM Architecture and Kernel Fusion (2025)

Caso Ilustrativo: Compresión de Llama-70B

Caso ilustrativo de arquitectura, no es una empresa real — Reducción de 140 GB a 40 GB VRAM

Llama de 70B: efecto de cuantizar con AWQ a INT4



Antes: 4× A100 (80 GB)
NVLink obligatorio · p4de.24xlarge (~$15 a $20/h, cifra a validar)

cuantizar

Después: 2× A10G o 1× A100
g5.12xlarge (~$5,60/h)

Comprimir antes de comprar: el ahorro supera el 65 % en este ejemplo

Fuente: Metodología de Dimensionamiento de Inferencia — Universidad de la Costa (2026)

Comparativa de Métodos de Cuantización

Matriz de selección arquitectónica según hardware disponible, formato de archivo y motor de inferencia

AWQ (INT4)

Enfoque Clave
Protege canales salientes críticos

Motor Óptimo
vLLM en GPUs cloud

Recomendación
Servidores de nube

GPTQ (INT4)

Enfoque Clave
Compensación analítica de error por capas

Motor Óptimo
AutoGPTQ / vLLM

Recomendación
Inferencia por lotes

GGUF (K-Quants)

Enfoque Clave
Cuantización mixta archivo autocontenido

Motor Óptimo
llama.cpp / Ollama

Recomendación
Edge y entorno local

FP8 Nativo

Enfoque Clave
Formato flotante de 8 bits en silicio

Motor Óptimo
TensorRT-LLM

Recomendación
Tarjetas Ada / Hopper

Fuente: Guía de Selección de Cuantización para Arquitectura de IA — CUC (2026)

SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

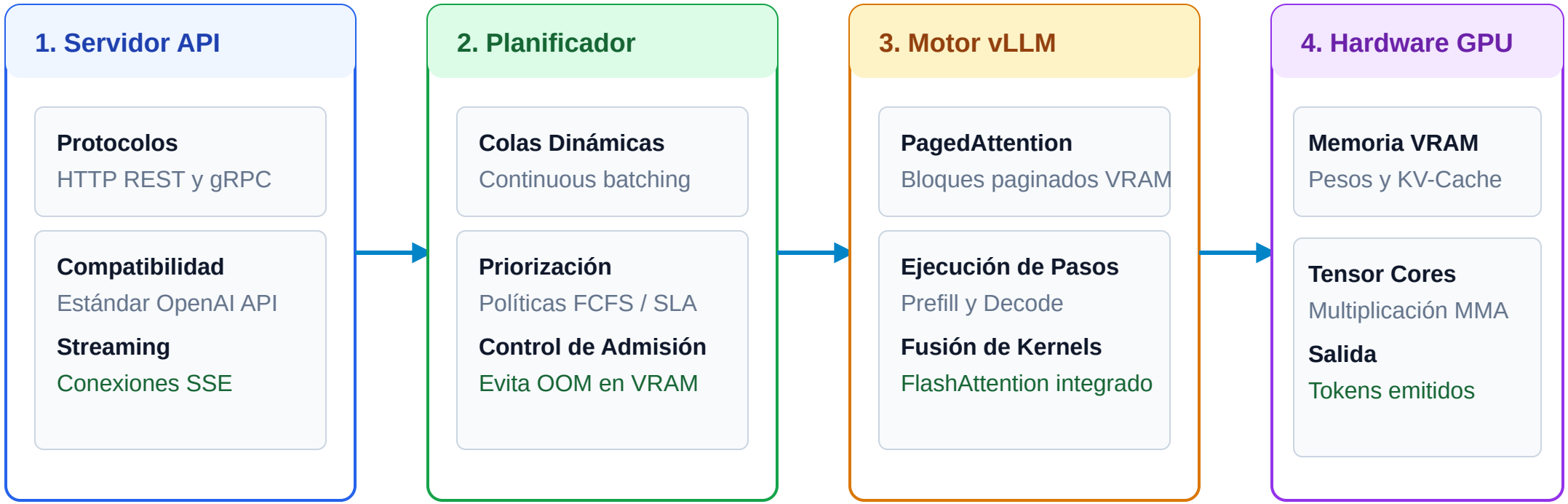
Parte 3 de 4 — Servidores de Inferencia Auto-Alojados (Self-Hosted)

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

Parte 3 de 4 — Servidores de Inferencia Auto-Alojados (Self-Hosted)

Anatomía de un Servidor de Inferencia Moderno

Desacoplamiento entre la capa de red, el planificador de lotes y el motor de ejecución en GPU

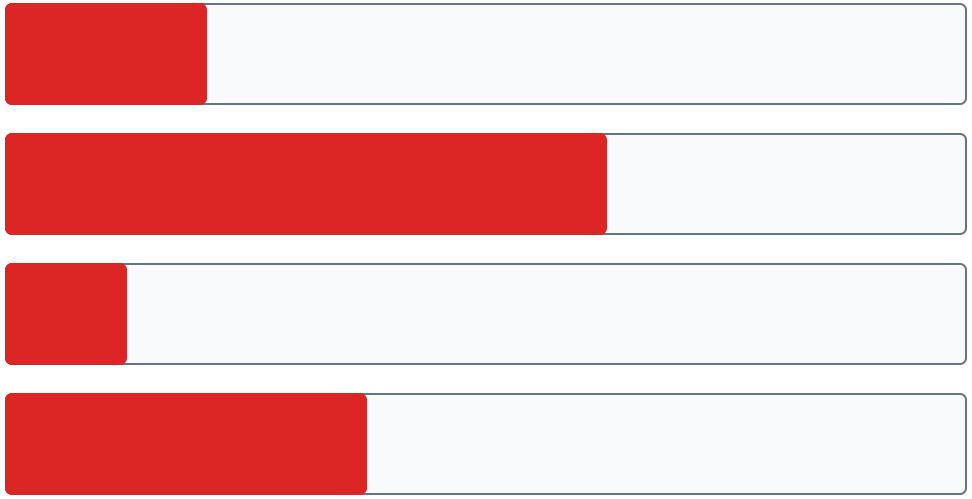


Fuente: Kwon et al. — Efficient Memory Management for Large Language Model Serving with PagedAttention (2023)

vLLM y PagedAttention: Cero Fragmentación

Inspirado en la memoria virtual paginada de los sistemas operativos para maximizar la VRAM

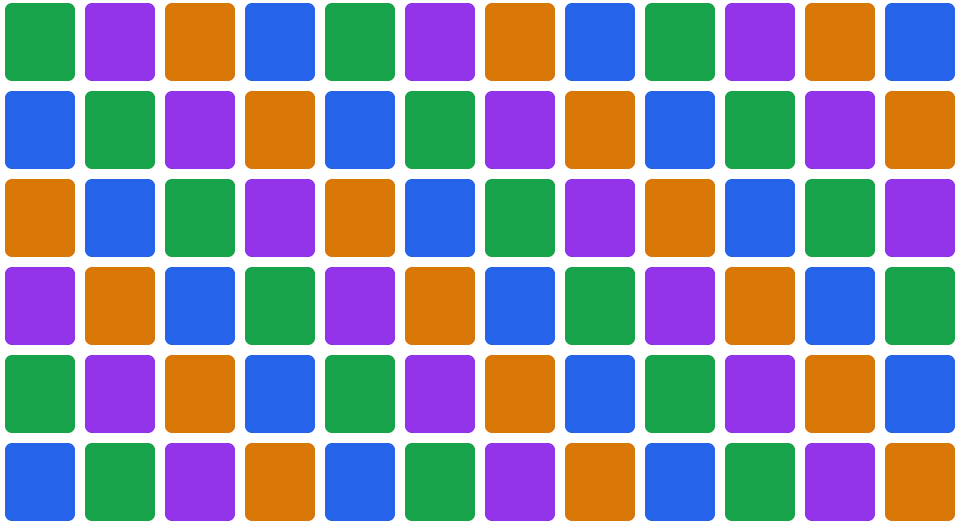
Asignación contigua



Se reserva todo el largo máximo

60 % a 80 % de memoria sin usar

PagedAttention



Bloques de 16 tokens + tabla de páginas

Cero fragmentación, hasta 4x throughput

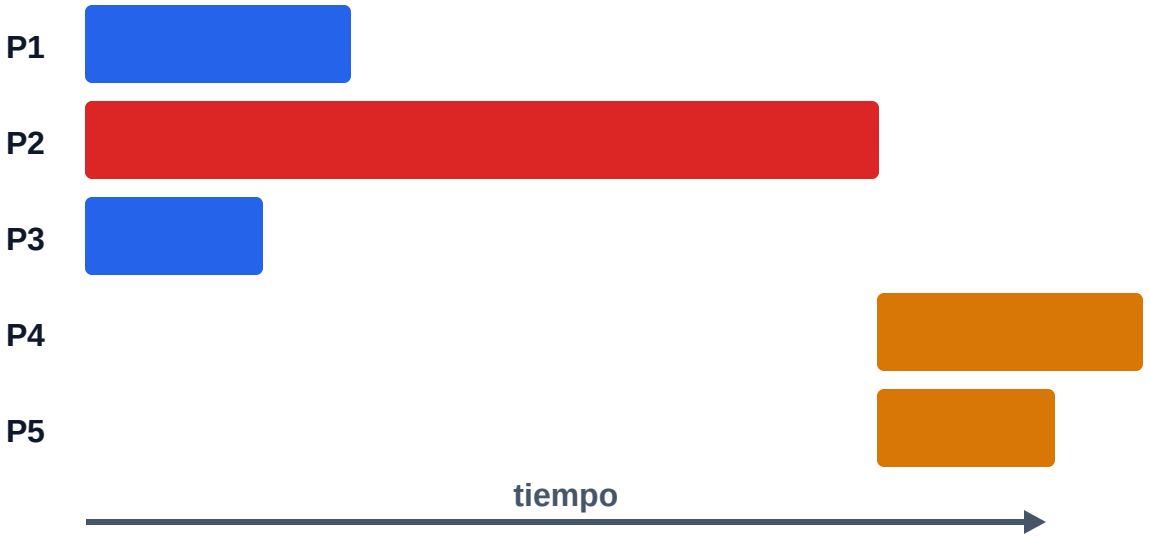
La misma idea de la memoria virtual, aplicada a la VRAM

Fuente: Kwon et al. — PagedAttention Paper, SOSP 2023 & UC Berkeley Sky Computing Lab (2024)

Continuous Batching (In-Flight Batching)

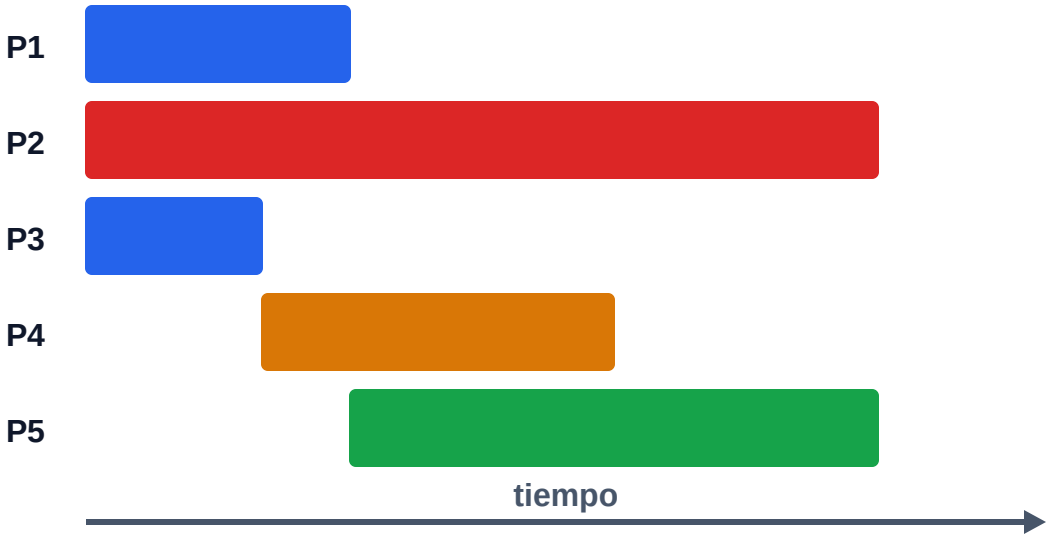
Inserción y retiro de peticiones a nivel de token individual en cada iteración del ciclo

Lote estático: esperan a la más larga



Latencia media y P99 suben

Continuous batching: se entra en cada token



Tensor Cores ocupados > 80 % del tiempo

Planificar por token, no por lote, elimina los huecos

Fuente: Yu et al. — Orca: A Distributed Serving System for Transformer-Based Generative Models (OSDI 2022 / vLLM 2024)

Decodificación Especulativa (Speculative Decoding)

Modelo borrador rápido propone candidatos y el modelo principal verifica en una pasada

1. Modelo Borrador (Draft)

Modelo Pequeño (ej. 1B)
Ejecución ultra-rápida y ligera

Emisión de K Candidatos
Propone una ráfaga de 3 a 5 tokens

Bajo Costo VRAM
Comparte GPU con modelo mayor

2. Verificación en Paralelo

Modelo Principal (70B)
Evalúa los K tokens simultáneamente

Cómputo en Prefill
Multiplica matrices en una pasada

Regla de Aceptación
Acepta tokens correctos en bloque

3. Rendimiento Resultante

Aceleración 2x a 3x
Emite múltiples tokens por ciclo

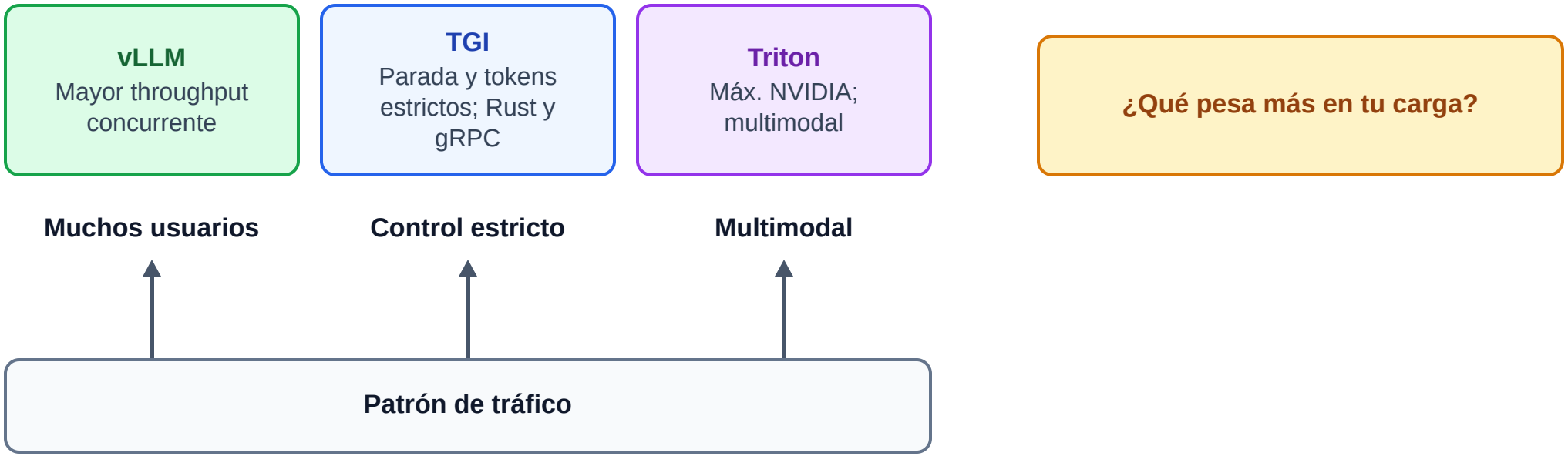
Cero Pérdida de Calidad
Distribución idéntica al modelo 70B

Ahorro en Hardware
Menor latencia sin cambiar tarjetas

Fuente: Leviathan et al. — Fast Inference from Transformers via Speculative Decoding (Google Research 2023)

Comparativa de Motores: vLLM, TGI y Triton

Evaluación de las plataformas de inferencia dominantes en despliegues auto-alojados

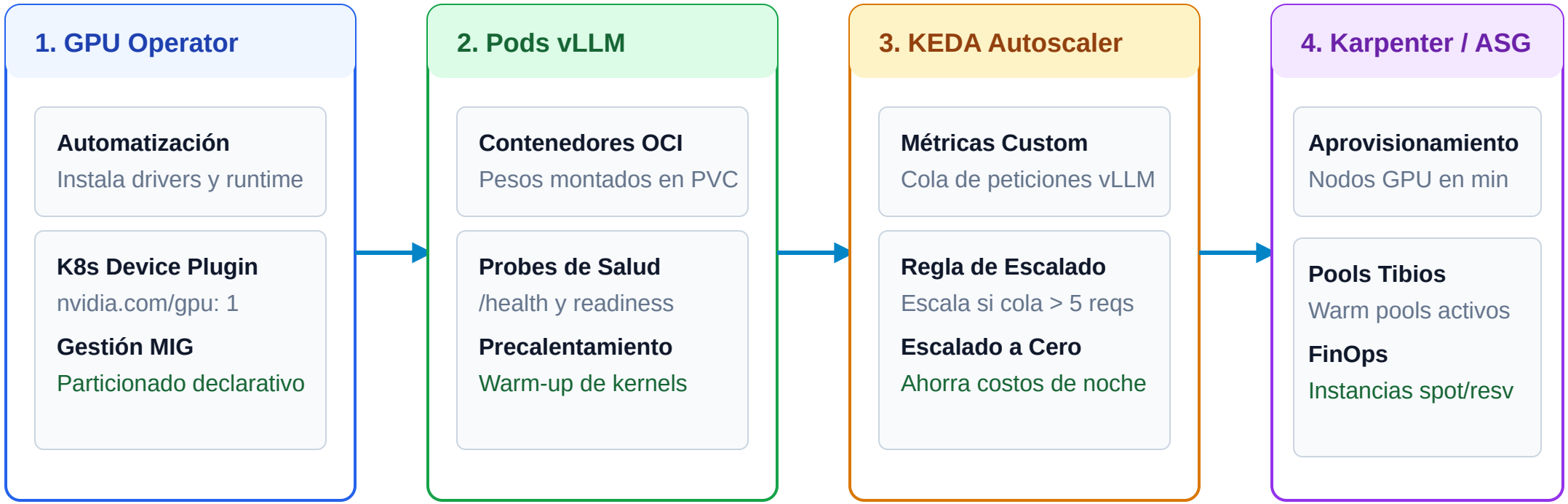


El motor se elige por el patrón de tráfico

Fuente: Benchmarks Oficiales de Rendimiento de Inferencia — MLCommons / Sky Computing (2025)

Despliegue en Kubernetes con GPU Operator y KEDA

Autoescalado horizontal basado en la profundidad de colas y utilización real de VRAM

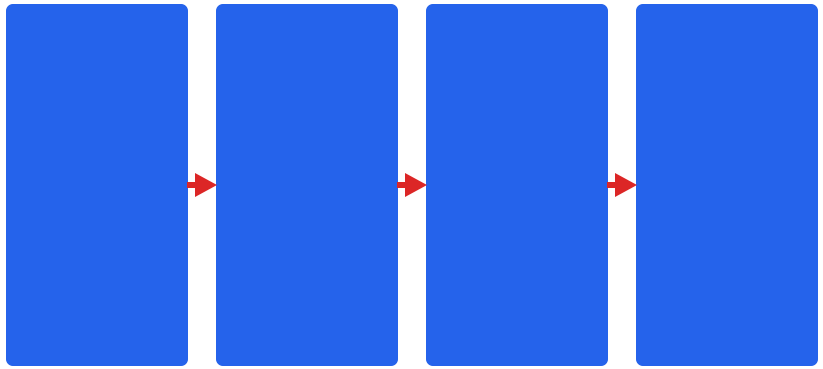


Fuente: CNCF — Kubernetes GPU Scheduling & KEDA AI Scaler Patterns (2025)

Paralelismo Tensorial y de Pipeline

Técnicas de partición de modelos masivos a través de múltiples GPUs y múltiples servidores

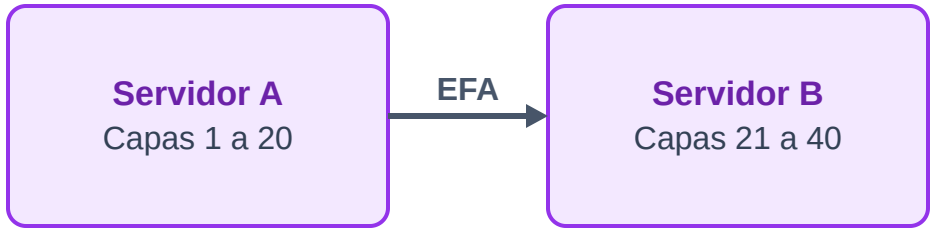
Tensorial: cada capa repartida



All-Reduce: exige NVLink (> 900 GB/s)

Dentro de un servidor, hasta 8 GPU

Pipeline: capas por servidor



Redes de 100/400 Gbps bastan

Riesgo: burbujas si no se satura el pipeline

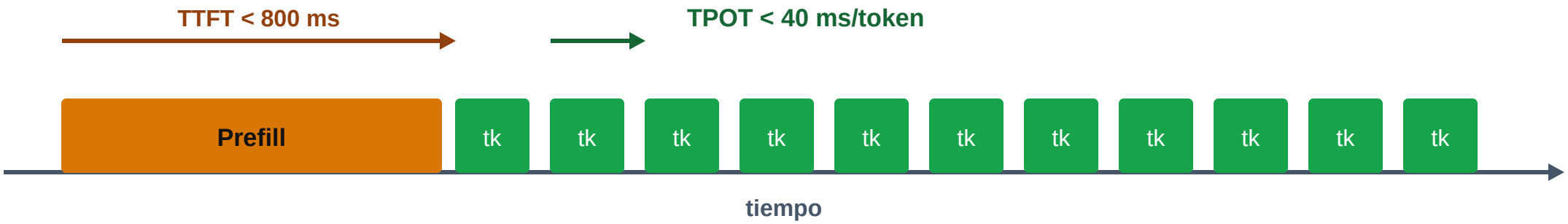
Si se parte la capa se necesita NVLink; si se parten las capas, basta la red

Fuente: Shoeybi et al. — Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism (2024)

Métricas de Servicio: TTFT, TPOT y Throughput

Los tres indicadores determinantes para definir acuerdos de nivel de servicio en inferencia

Vida de una petición en streaming



Throughput global > 1.500 tokens/seg por nodo GPU
Es lo que fija el costo real por token

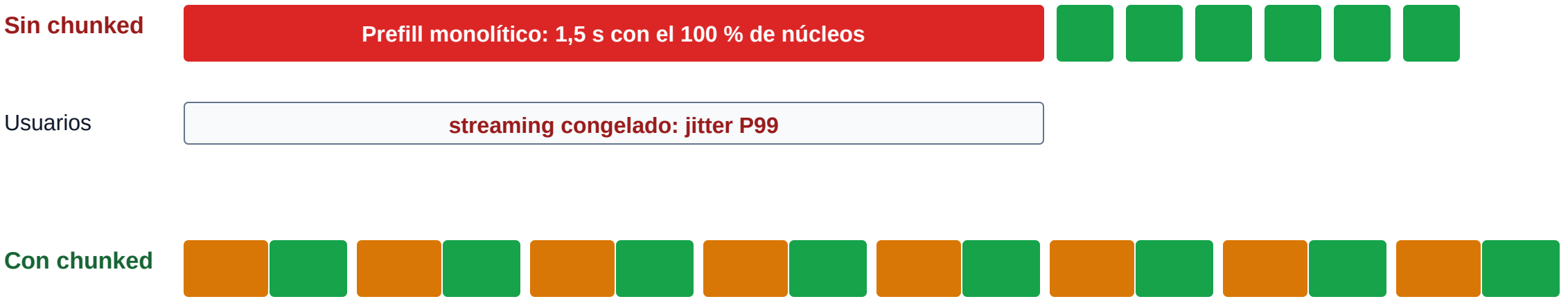
TTFT lo siente quien espera; TPOT, quien lee; el throughput, quien paga

Fuente: AWS Generative AI Lens — SLA and Performance Tuning for LLMs (2025)

Control de Latencia P99: Chunked Prefill

Partición de prompts gigantescos en bloques para evitar congelamientos en peticiones concurrentes

Prompt largo de 16k tokens mientras otros usuarios hacen streaming



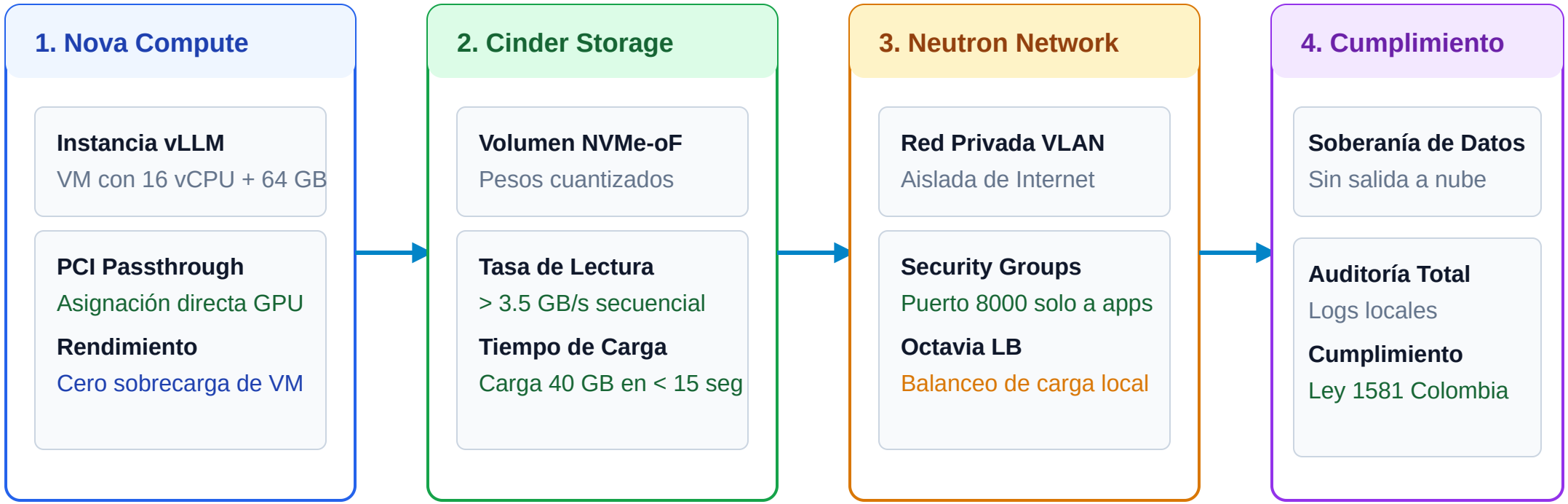
Amarillo: trozo de 512 tokens · Verde: decodificación de los demás

Trocear el prompt reparte la GPU y evita las pausas

Fuente: Agrawal et al. — Sarathi: Efficient LLM Inference by Chunked-Prefills (2024)

vLLM en Nube Privada OpenStack

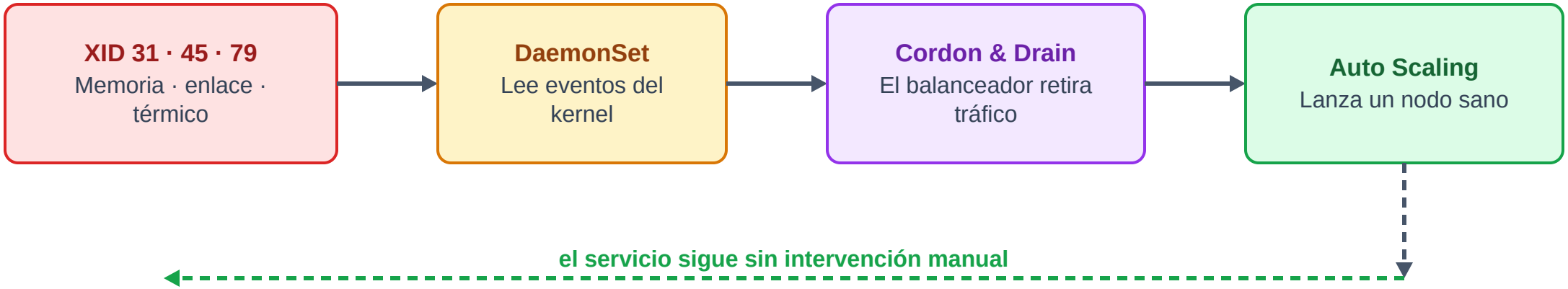
Aprovisionamiento de servidores de inferencia sobre hipervisores KVM con almacenamiento Cinder



Fuente: Guía de Arquitectura de IA en OpenStack — Laboratorio CUC (2026)

Recuperación de Fallos y Errores XID de GPU

Detección temprana de anomalías en silicio y aislamiento automático de nodos degradados



Un XID no se depura en caliente: el nodo se reemplaza

Fuente: NVIDIA Developer — GPU XID Error Code Reference Guide (2025)

Auditoría de Tokens y FinOps en IA

Control de costos unitarios y atribución de consumo por centro de costos en servidores propios

1. Atribución de Consumo

Etiquetado por API Key
Identifica usuario y departamento

Métricas en Gateway
Mide prompt y completion tokens

Showback / Chargeback
Imputación presupuestal mensual

2. Cálculo de Costo Unitario

Fórmula de Costo Real
Costo Servidor / Millones Tokens

Tasa de Ocupación
A mayor concurrencia, menor costo

Comparativa con APIs
Valida la rentabilidad del hardware

3. Prevención de Abuso

Cuotas por Aplicación
Techos máximos de tokens al mes

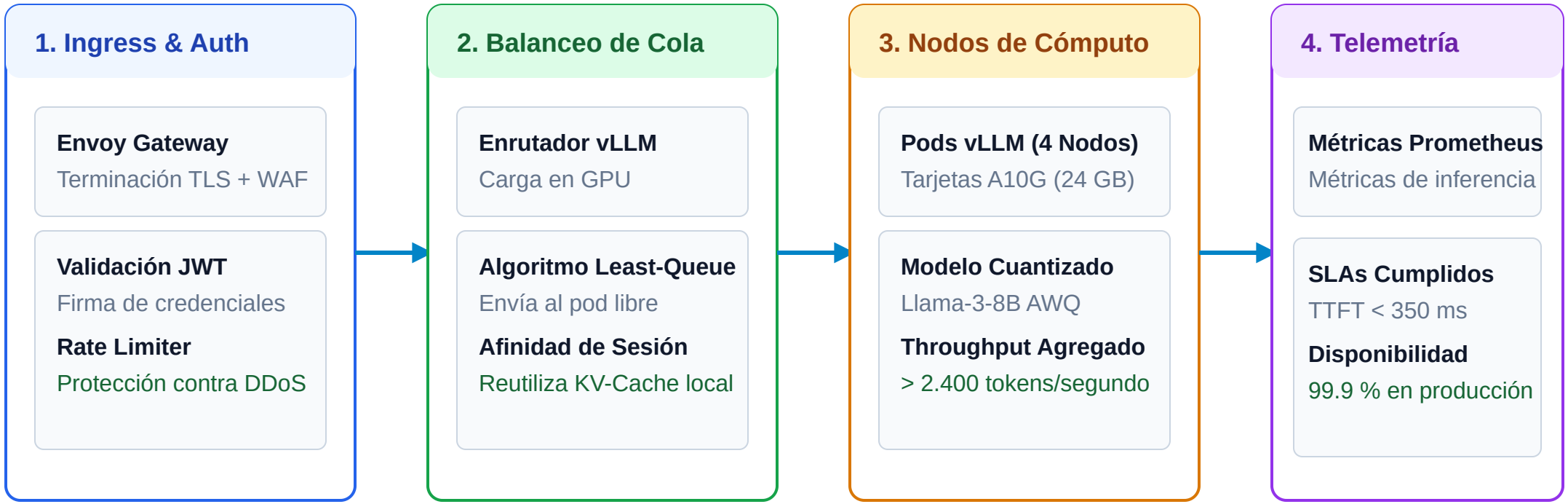
Detección de Bucles
Frena agentes en bucles infinitos

Alarmas en Tiempo Real
Notificación ante picos anómalos

Fuente: FinOps Foundation — AI and Machine Learning FinOps Framework (2025)

Caso Ilustrativo: Arquitectura Self-Hosted de Producción

Caso ilustrativo de arquitectura, no es una empresa real — Flujo integral de inferencia en clúster privado



Fuente: Patrón de Referencia de Servidores de Inferencia en Producción — CUC (2026)

CHECKLIST DE OPERACIONES

Decálogo de Operación para Inferencia Self-Hosted

Buenas prácticas indispensables para mantener servidores de inferencia estables y económicos

1. Memoria VRAM

Regla del 90 %

Fijar límite seguro de memoria para evitar Out-Of-Memory en el runtime CUDA

gpu_memory_util: 0.90

2. Planificación

Continuous Batch

Habilitar siempre colas dinámicas y Chunked Prefill para estabilizar P99

PagedAttention On

3. Telemetría

DCGM + KEDA

Monitorear errores XID, temperatura y profundidad de cola para autoescalar

Alertas Proactivas

4. Resiliencia

Drenado Automático

Aislar tarjetas con fallos de hardware y contar con pools tibios de respaldo

Alta Disponibilidad

Fuente: Síntesis de Operaciones de IA — Universidad de la Costa (2026)

SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

Parte 4 de 4 — Servicios Gestionados vs Auto-Alojados y TCO

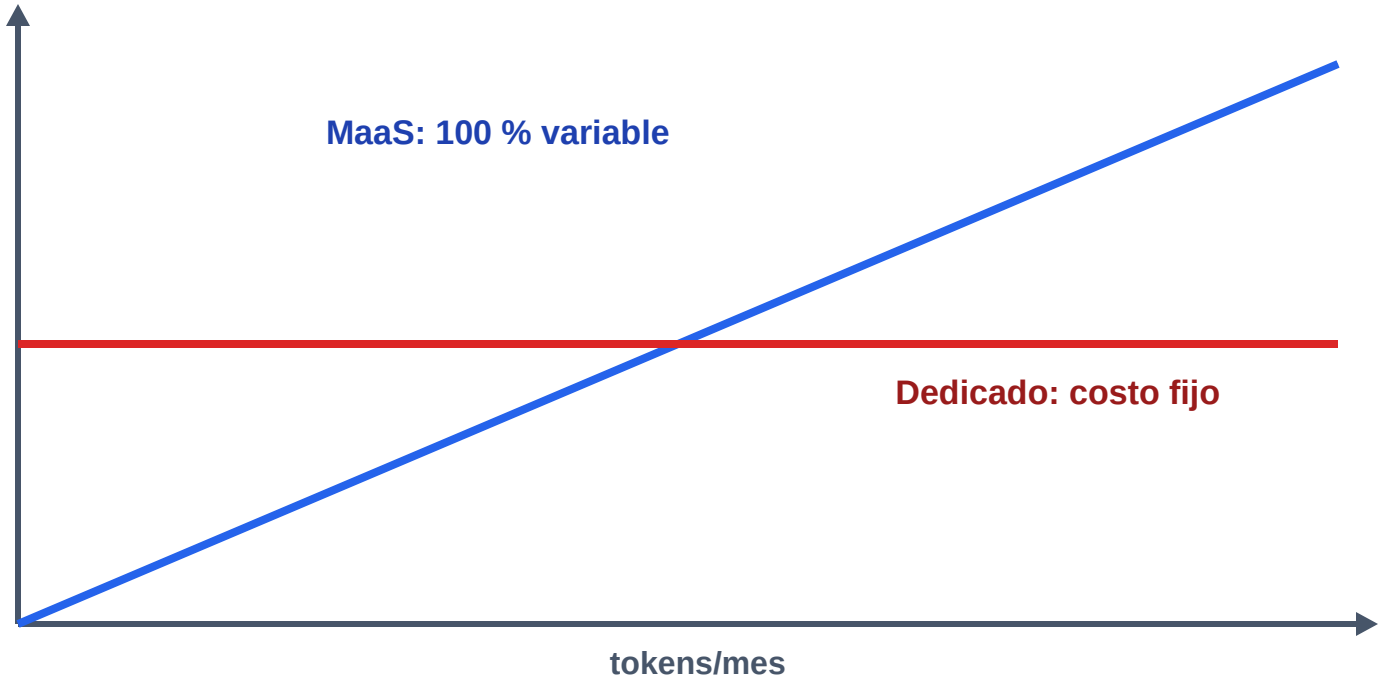
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

Parte 4 de 4 — Servicios Gestionados vs Auto-Alojados y TCO

El Dilema: Gestionado frente a Auto-Alojado

Comparativa de responsabilidades, costos fijos vs variables y tiempo de salida al mercado

Costo mensual según el volumen (esquema, no a escala)



Bedrock
Operación mínima y acceso inmediato a modelos de frontera

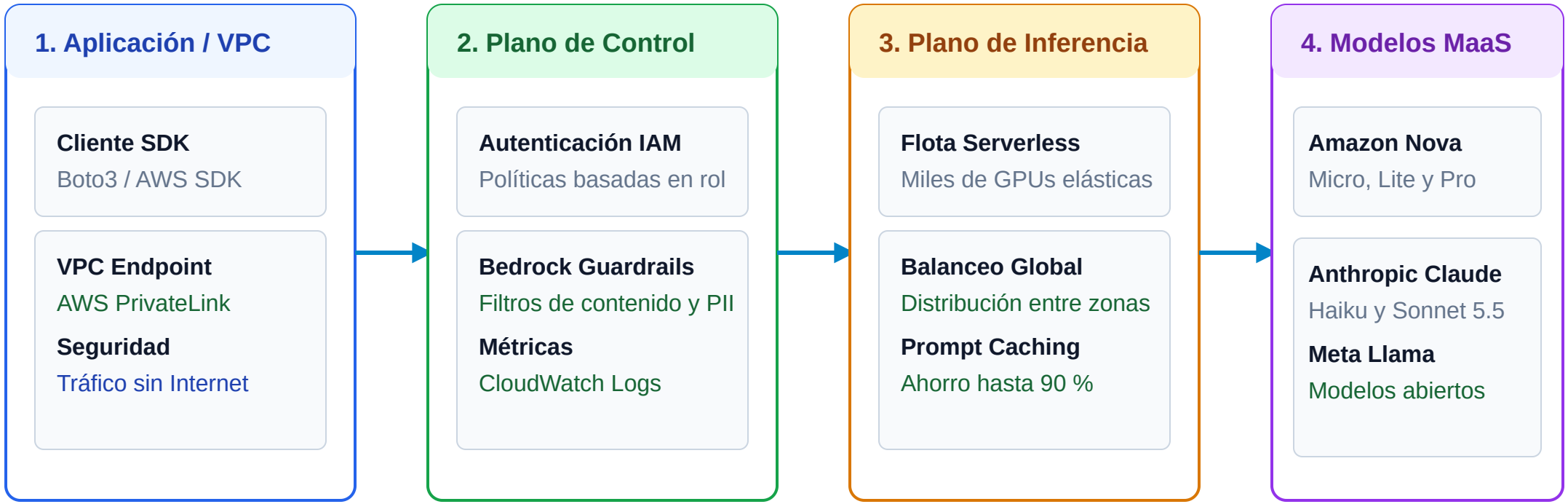
Servidor propio
Control total; hay que operar CUDA, vLLM y fallos XID

No es solo costo: también pesan operación, tiempo de salida y privacidad

Fuente: AWS Prescriptive Guidance — Choosing Between Managed and Self-Hosted LLMs (2025)

Arquitectura de Amazon Bedrock

Punto de acceso serverless unificado con aislamiento multi-tenant y sin gestión de servidores



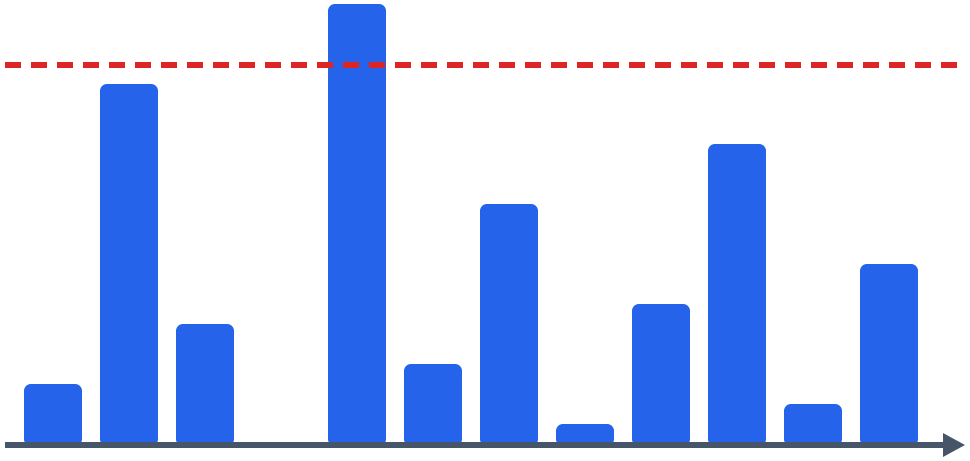
Fuente: AWS Documentation — Amazon Bedrock User Guide & Security Architecture (2025)

On-Demand frente a Capacidad Aprovisionada

Pago elástico por token individual frente a reserva de unidades de rendimiento (PTUs)

On-Demand: se paga por token

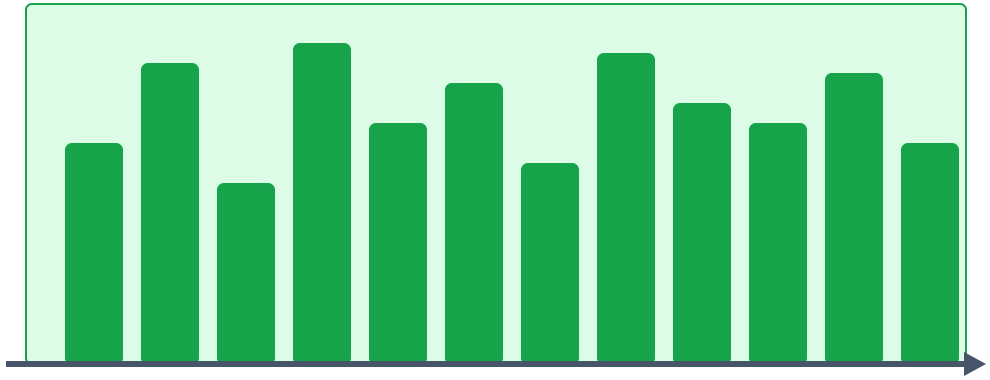
Cuota TPM compartida (línea roja)



Escala a cero, sin compromiso

Aprovisionada: tarifa por hora

Throughput reservado (Model Unit)



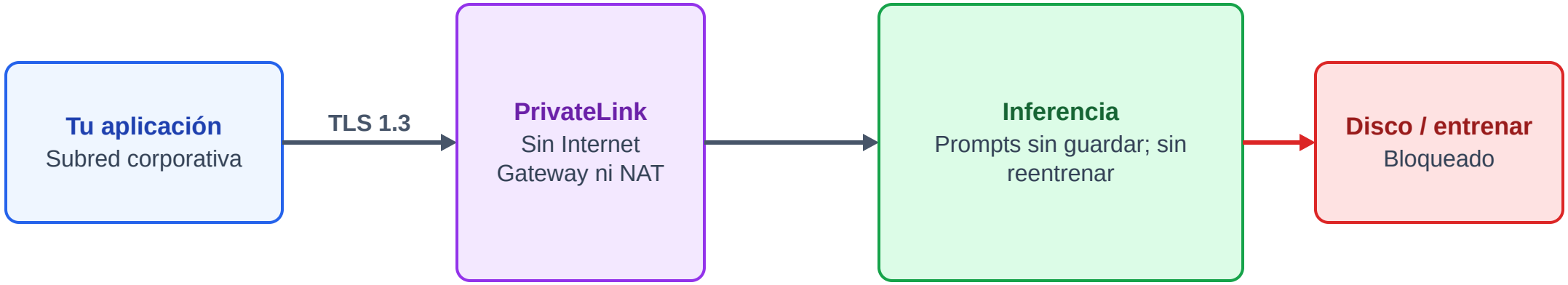
Compromiso de 1 o 6 meses en modelos propios

Tráfico irregular: On-Demand. Tráfico sostenido y latencia crítica: aprovisionar

Fuente: AWS Bedrock Documentation — Provisioned Throughput Guidelines (2025)

Aislamiento y Retención Cero de Datos (ZDR)

Garantías contractuales y técnicas de que los datos no se utilizan para entrenar modelos



Control con IAM · Cifrado en reposo con KMS

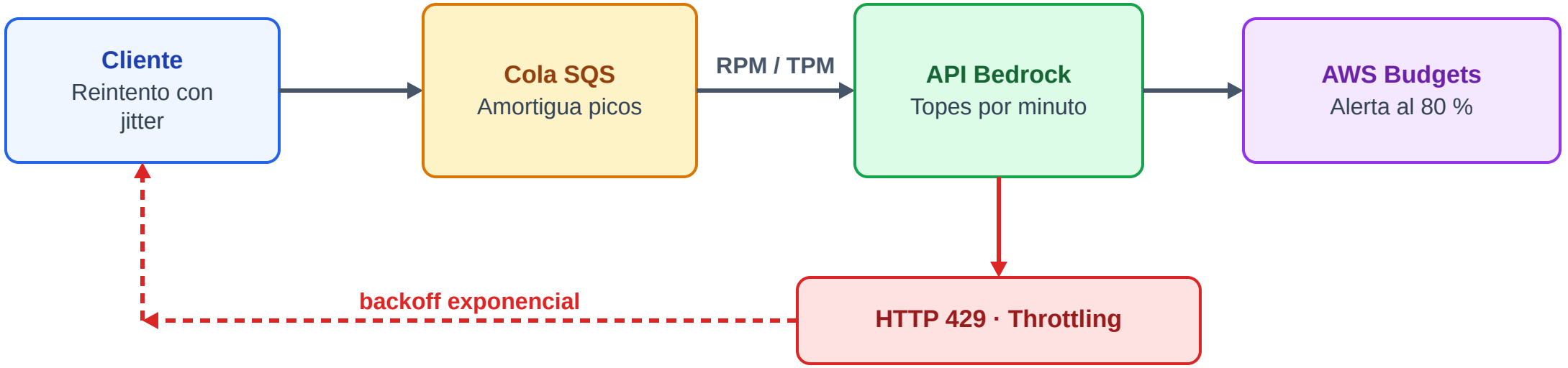
Certificaciones: SOC 1/2/3 · ISO 27001 · HIPAA · PCI-DSS

La confianza se construye con controles verificables en cada tramo

Fuente: AWS Bedrock Security & Data Protection Policy (2025)

Gestión de Cuotas y Límites de Tasa en MaaS

Políticas de protección contra costos imprevistos y estrangulamiento de peticiones (Throttling)



El 429 no es un error raro: es parte del diseño

Fuente: AWS Bedrock Quotas and Service Limits Documentation (2025)

Responsabilidad Compartida en Inteligencia Artificial

Delimitación clara entre el proveedor de nube, el creador del modelo y el arquitecto consumidor

Cliente / arquitecto

Datos y prompts · filtros contra prompt injection · verificar salidas

Proveedor del modelo

Curaduría de datos de preentrenamiento · alineación · integridad de la arquitectura

Proveedor de nube (AWS)

Seguridad física · aislamiento de red y virtualización · SLA de la API

Contratar un modelo no traslada la responsabilidad sobre los datos ni las salidas

Fuente: AWS Shared Responsibility Model for Generative AI (2025)

Estructura de Precios Reales en Amazon Bedrock

Tarifas por millón de tokens en región US-East (N. Virginia), vigentes a noviembre de 2026



Largo de barra = precio de salida: el salto entre modelos llega a 70 veces

Fuente: AWS Bedrock Official Pricing (Consultado Noviembre 2026)

Punto de Equilibrio: Gestionado frente a GPU

Fórmula matemática para determinar el volumen de tokens mensual que justifica un servidor dedicado

Cruce: costo fijo ÷ costo medio por token

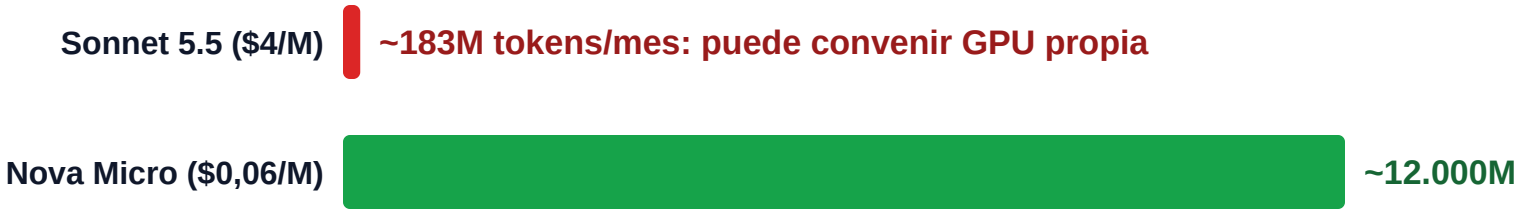
g5.xlarge ≈ $734/mes
$1,006/h × 730 h

÷

Costo medio del token API
Mezcla 3 entrada : 1 salida

=

Volumen de cruce



Sigue pesando la operación: esta cuenta no incluye equipo ni modelo equivalente

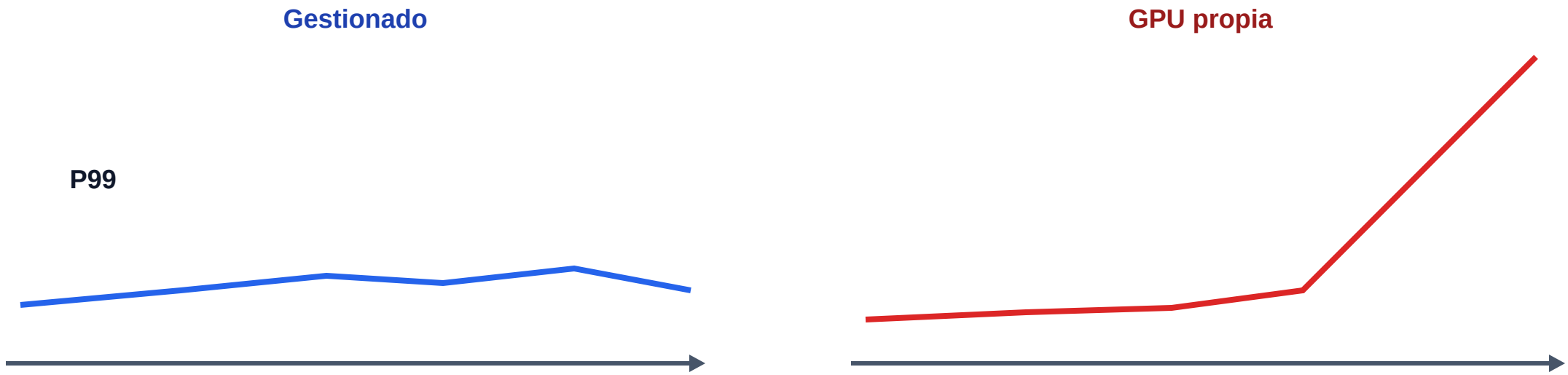
Justificar GPU propia solo si la utilización sostenida supera el 70 % diario

Fuente: Metodología de FinOps para Arquitectura de IA — CUC (2026)

Perfil de Latencia y SLAs: Gestionado vs GPU

Comportamiento del tiempo de respuesta bajo ráfagas de tráfico y fallos de zona

Comportamiento ante una ráfaga (esquema)



El SLA gestionado lo respalda un contrato; el propio, tu diseño del clúster

Fuente: AWS Service Level Agreements & Real-world AI Benchmark Analysis (2025)

Mitigación del Bloqueo de Proveedor (Vendor Lock-in)

Capas de abstracción y adopción de la especificación OpenAI API para desacoplar el código



Fuente: The Open-Source AI Gateway Framework & LiteLLM Documentation (2025)

Ruteo Dinámico Inteligente de Inferencia

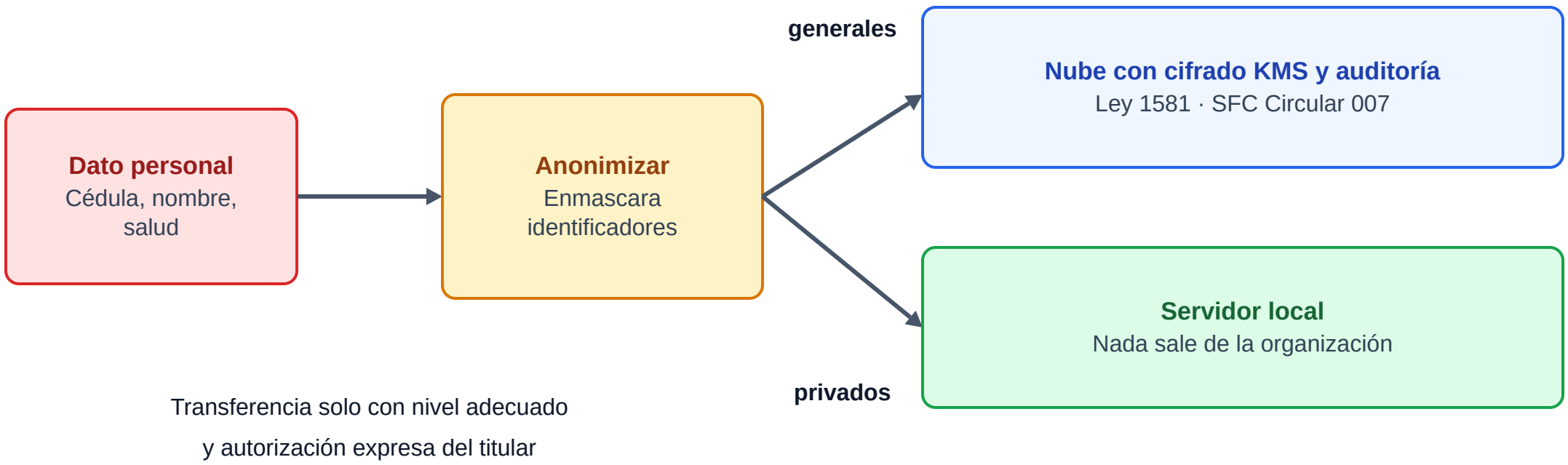
Enrutamiento selectivo por complejidad: modelos ligeros para el 70 % y modelos avanzados para el resto



Fuente: RouteLLM Framework & Berkeley Sky Computing Lab (2024)

Soberanía de Datos y Cumplimiento en Colombia

Exigencias regulatorias de la Ley 1581 de 2012 y Circulares de la Superfinanciera para IA



La ley no prohíbe la nube: exige saber qué dato va a dónde

Fuente: Ley 1581 de 2012 — Congreso de Colombia & Circular Externa 007 de la SFC (2025)

GUÍA DE DECISIÓN

Matriz de Decisión: ¿Gestionado o Auto-Alojado?

Árbol de criterios arquitectónicos para seleccionar la estrategia de despliegue óptima

API Gestionada (Bedrock)

Perfil de Proyecto

Fase inicial, pilotos, RAG o tráfico esporádico

Ventaja Clave

Cero costo de mantenimiento y acceso a modelos frontera

Elección Óptima < 50M tok/mes

Arquitectura Híbrida (Router)

Perfil de Proyecto

Aplicaciones a gran escala con tráfico heterogéneo

Ventaja Clave

Despacha 70 % a modelo bajo costo y 30 % a frontera

Mayor ROI en producción

Auto-Alojado (vLLM)

Perfil de Proyecto

Datos ultra-sensibles, soberanía o tráfico masivo

Ventaja Clave

Control total de pesos, latencia y costo fijo

Obligado por regulación

Fuente: Guía de Decisión de Inferencia en Nube — Universidad de la Costa (2026)

Conclusiones y Conexión con la Semana 16

De la infraestructura de cómputo y compresión hacia las capacidades avanzadas de agentes y razonamiento

1. Silicio y Memoria

Hardware Acelerado

El ancho de banda HBM3 y los enlaces NVLink determinan el techo de latencia

Hopper / Blackwell

2. Cuantización

Compresión Eficiente

Técnicas AWQ y GPTQ junto con GQA reducen la VRAM 4x sin perder precisión

INT4 y FP8 Estándar

3. Motores vLLM

Inferencia de Nube

PagedAttention y continuous batching maximizan la ocupación del hardware dedicado

Throughput Escala

4. Próxima Semana

Semana 16: IA III

Capacidades avanzadas Razonamiento profundo, programación, agentes y repaso de Unidad 3

Hacia la Autonomía

Fuente: Síntesis de Arquitectura en la Nube — Universidad de la Costa (2026)